

Case Study: KL ERP System

1. Introduction

Universities and colleges manage a wide range of academic and administrative processes — student admissions, course registration, attendance, examinations, and fee collection — that are difficult to coordinate reliably using manual or fragmented systems. This case study presents the design and development of the KL ERP System, a web-based Enterprise Resource Planning platform for academic institutions that brings students, faculty, and administrative staff onto a single integrated platform.

The problem statement requires selecting a real-world case study, defining user stories, choosing an appropriate software development process model and programming language, and proposing an implementation and evaluation strategy. The KL ERP System addresses common institutional challenges such as manual attendance registers, delayed result publication, fragmented fee records, and lack of real-time visibility into academic progress, by digitizing and automating these core workflows.

Core Objectives

- Provide a single platform for course registration, attendance, and examinations.
- Give students real-time visibility into attendance, results, and fee status.
- Enable faculty to manage attendance and results digitally.
- Streamline fee collection with online payment and receipts.
- Support administrators with consolidated academic and financial reporting.

2. User Stories

User ID	User Story	Priority
US-01	As a student, I want to log in securely so that I can access my academic records.	High
US-02	As a faculty member, I want to log in securely to manage my classes.	High
US-03	As a student, I want to register for courses each semester.	High
US-04	As a faculty member, I want to mark student attendance for each class.	High
US-05	As a faculty member, I want to upload exam marks and grades.	High
US-06	As a student, I want to view my attendance percentage per subject.	High
US-07	As a student, I want to view my exam results and grade card.	High
US-08	As a student, I want to pay semester fees online and download a receipt.	High
US-09	As an admin, I want to manage student records and enrollments.	Medium
US-10	As an admin, I want to manage faculty records and course allocations.	Medium
US-11	As an admin, I want to manage the academic timetable.	Medium
US-12	As an admin, I want to generate academic and financial reports.	Medium

3. Software Development Process Model

Selected Model: Agile Scrum

Reason for Selection

- Requirements evolve across academic departments and semesters.
- Continuous feedback from students, faculty, and administration.

- Incremental delivery of modules (registration, attendance, exams, fees).
- Faster development and release cycles aligned with semester timelines.
- Easier to isolate and fix bugs per module.
- Well suited for large, multi-user web-based institutional applications.

Sprint Plan

Sprint	Features
Sprint 1	User Authentication & Role Management
Sprint 2	Student & Faculty Profile Management
Sprint 3	Course Registration & Timetable
Sprint 4	Attendance Management
Sprint 5	Examination & Results Management
Sprint 6	Fee Payment & Financial Reporting
Sprint 7	Testing & Deployment

4. Programming Language and Technologies

Backend

- Java (Spring Boot)

Reasons

- Secure and scalable
- Well suited for enterprise-level applications
- Strong REST API support
- Excellent database connectivity and transaction management

Frontend

- HTML5
- CSS3
- JavaScript
- React.js

Database

- MySQL

Other Technologies

- REST API
- JWT Authentication
- Razorpay/Stripe Payment Gateway (fee payments)
- Git
- Docker
- AWS Cloud

5. Implementation Strategy

Phase 1 — Requirement Analysis

- Gather requirements from students, faculty, and administrative departments.
- Create Software Requirement Specification (SRS).
- Prepare use case diagrams.

Phase 2 — System Design

- Design database schema (students, faculty, courses, attendance, results, fees).
- Design UI wireframes for student and faculty portals.
- Create UML diagrams.
- Develop system architecture.

Phase 3 — Development

- Implement user authentication and role management.
- Develop student and faculty profile modules.
- Build course registration and timetable management.
- Build attendance marking and reporting.
- Build examination and results management.
- Integrate online fee payment gateway.

Phase 4 — Testing

Unit Testing

- Login and role validation
- Attendance percentage calculations
- Grade computation logic

Integration Testing

- Course registration + timetable
- Attendance + result eligibility

System Testing

- Complete application testing end-to-end

User Acceptance Testing (UAT)

- Verify business requirements with students, faculty, and administration

Phase 5 — Deployment

- Docker Container
- AWS EC2
- MySQL Database
- Nginx Web Server

6. Evaluation Strategy

Evaluation Parameter	Method
Functional Accuracy	Requirement verification against SRS
Performance	Load testing using JMeter during course registration peak periods
Security	Authentication, role-based access, and penetration testing

Evaluation Parameter	Method
Reliability	Continuous monitoring and data consistency checks
Usability	User feedback survey from students and faculty
Scalability	Concurrent user testing during registration and exam result release
Maintainability	Code review, modular design, and documentation

8. UML Diagrams

8.1 Use Case Diagram

The use case diagram below captures the primary interactions between the two main actor groups — Students, who use the system for registration, attendance, results, and fee payments, and Faculty/Admin, who manage academic and administrative operations — with the core functions of the KL ERP System.

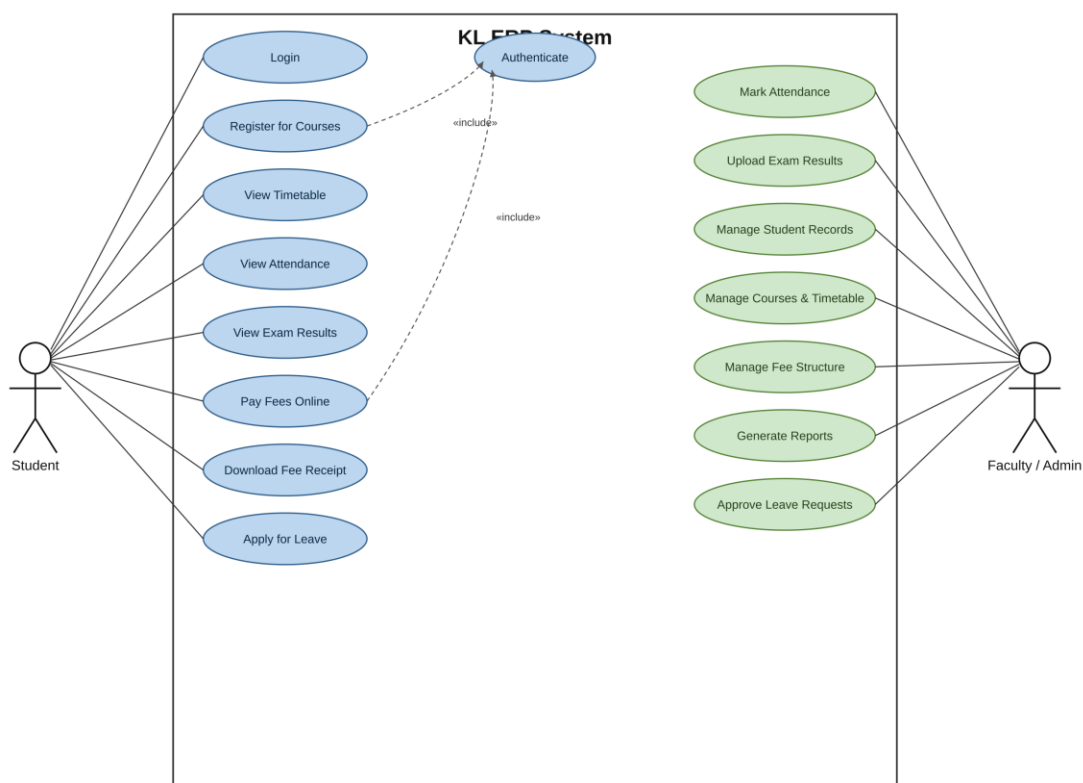
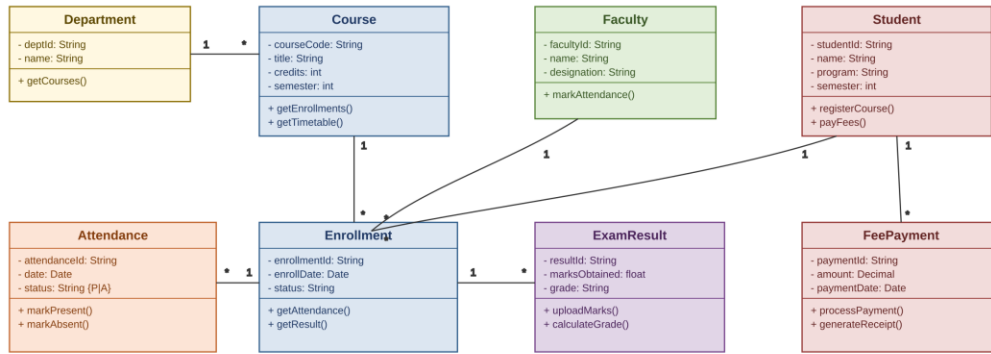


Figure 1: Use Case Diagram — KL ERP System

8.2 Class Diagram

The class diagram below models the core domain entities of the system — Department, Course, Faculty, Student, Enrollment, Attendance, ExamResult, and FeePayment — along with their key attributes, operations, and relationships.



Class Diagram — Core domain model of the KL ERP System

Figure 2: Class Diagram — KL ERP System

9. Conclusion

The proposed KL ERP System addresses the operational challenges of academic institutions by combining a scalable Spring Boot/React architecture with an Agile Scrum delivery approach. The defined user stories, sprint plan, and UML models provide a clear blueprint for incremental development, while the phased implementation and evaluation strategy ensure that functional accuracy, performance, security, and usability are validated before deployment. Once implemented, the system is expected to reduce manual administrative overhead, give students and faculty real-time access to academic information, and provide administrators with reliable, consolidated reporting — forming a solid foundation for future enhancements such as mobile access, analytics dashboards, and integration with learning management systems.